

Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning technique that enables an agent to learn in an interactive environment **by trial and error using feedback from its own actions and experiences**. For each good action, the agent gets positive feedback, and for each bad action, the agent gets negative feedback or penalty.

To know the meaning of reinforcement learning, let's go through the formal definition.

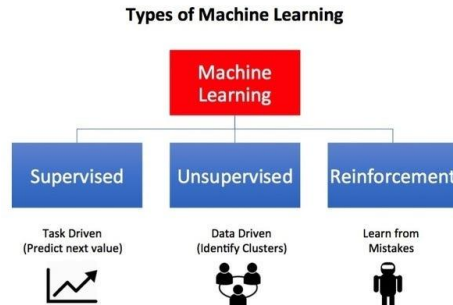
Reinforcement learning, a type of machine learning, in which agents take actions in an environment aimed at maximizing their cumulative rewards – **NVIDIA**

Reinforcement learning (RL) is based on rewarding desired behaviors or punishing undesired ones. Instead of one input producing one output, the algorithm produces a variety of outputs and is trained to select the right one based on certain variables – **Gartner**

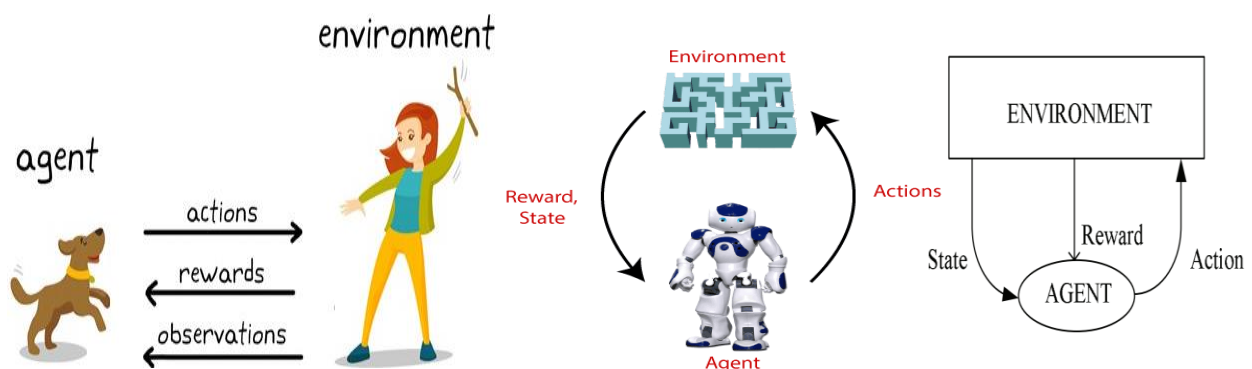
It is a type of machine learning technique where a computer agent learns to perform a task through repeated trial and error interactions with a dynamic environment. This learning approach enables the agent to make a series of decisions that maximize a reward metric for the task without human intervention and without being explicitly programmed to achieve the task – **Mathworks**

Explanation to Reinforcement Learning

In Reinforcement Learning, the agent learns automatically using feedbacks without any labeled data, unlike supervised learning. Since there is no labeled data, so the agent is bound to learn by its experience only. It is unsupervised learning.



RL solves a specific type of problem where decision making is sequential, and the goal is long-term, such as game-playing, robotics, etc. The agent interacts with the environment and explores it by itself. The primary goal of an agent in reinforcement learning is to improve the performance by getting the maximum positive rewards. The agent learns with the process of hit and trial, and based on the experience, it learns to perform the task in a better way. Hence, we can say that **"Reinforcement learning is a type of machine learning method where an intelligent agent (computer program) interacts with the environment and learns to act within that."**



We can see a dog and a master (from first picture). Let's imagine you are training your dog to get the stick. Each time the dog gets a stick successfully, you offered him a feast (a bone let's say). Eventually, *the dog understands the pattern, that whenever the master throws a stick, it should get it as early as it can to gain a reward (a bone) from a master in a lesser time*. We can rethink that- How a Robotic dog learns the movement of his arms is an example of Reinforcement learning.

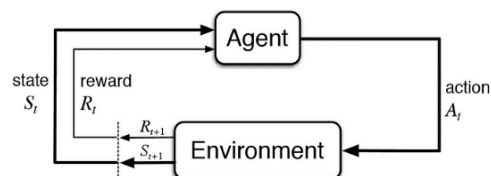
It is a core part of Artificial intelligence, and all AI agent works on the concept of reinforcement learning. Here we do not need to pre-program the agent, as it learns from its own experience without any human intervention.

Example: Suppose there is an AI agent present within a maze environment, and his goal is to find the diamond. The agent interacts with the environment by performing some actions, and based on those actions, the state of the agent gets changed, and it also receives a reward or penalty as feedback.

The agent continues doing these three things (take action, change state/remain in the same state, and get feedback), and by doing these actions, he learns and explores the environment.

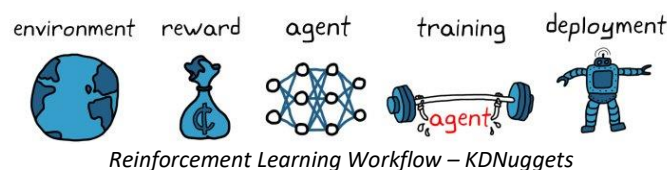
The agent learns that what actions lead to positive feedback or rewards and what actions lead to negative feedback penalty. As a positive reward, the agent gets a positive point, and as a penalty, it gets a negative point.

Though both **supervised and reinforcement learning** use mapping between input and output, unlike supervised learning where the feedback provided to the agent is correct set of actions for performing a task, *reinforcement learning uses rewards and punishments as signals for positive and negative behavior.*



As **compared to unsupervised learning**, **reinforcement learning** is different in terms of goals. While the goal in unsupervised learning is to find similarities and differences between data points, ***in the case of reinforcement learning the goal is to find a suitable action model that would maximize the total cumulative reward of the agent.***

Reinforcement Learning Workflow:



- Create the Environment
- Define the reward
- Create the agent
- Train and validate the agent
- Deploy the policy

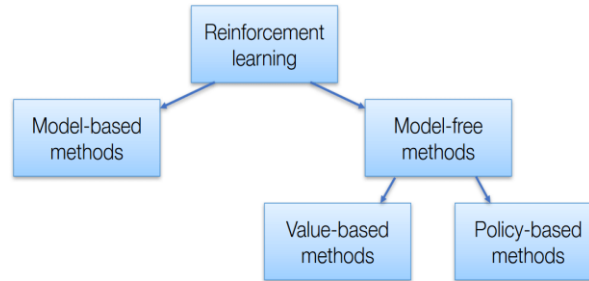
Key Features of Reinforcement Learning

- Agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward (Feedback isn't prompt but delayed)
- Environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.
- Decision making is sequential
- Time plays a major role in reinforcement problems

- The following data it receives is determined by the agent's actions

Approaches to implement Reinforcement Learning

There are mainly **three ways to implement reinforcement-learning in ML**, which are:



Value-based: The value-based approach is about to **find the optimal value function, which is the maximum value at a state under any policy**. Therefore, the agent expects the long-term return at any state(s) under policy π .

Policy-based: Policy-based approach is to **find the optimal policy for the maximum future rewards without using the value function**. In this approach, *the agent tries to apply such a policy that the action performed in each step helps to maximize the future reward*. The policy-based approach has mainly two types of policy:

- **Deterministic:** The same action is produced by the policy (π) at any state.
- **Stochastic:** In this policy, probability determines the produced action.

Model-based: In the model-based approach, **a virtual model is created for the environment, and the agent explores that environment to learn it**. There is no particular solution or algorithm for this approach because the model representation is different for each environment.

Types of Reinforcement learning

There are mainly two types of reinforcement learning, which are:

- Positive Reinforcement
- Negative Reinforcement

Positive Reinforcement: The positive reinforcement learning means **adding something to increase the tendency** that expected behavior would occur again. It impacts positively on the behavior of the agent and increases the strength of the behavior. *This type of reinforcement can sustain the changes for a long time, but too much positive reinforcement may lead to an overload of states that can reduce the consequences.*

Advantages:

- Maximizes the performance of an action
- Sustain change for a longer period

Disadvantage

- Excess reinforcement can lead to an overload of states which would minimize the results.

Negative Reinforcement: The negative reinforcement learning is opposite to the positive reinforcement **as it increases the tendency that the specific behavior will occur again by avoiding the negative condition**. It can be more effective than the positive reinforcement depending on situation and behavior, but it provides reinforcement only to meet minimum behavior.

Advantages

- Maximized behavior
- Provide a decent to minimum standard of performance

Disadvantage

- It just limits itself enough to meet up a minimum behavior

Elements of Reinforcement Learning

There are four main elements of Reinforcement Learning, which are given below:

- Policy
- Reward Signal
- Value Function
- Model of the environment

1) Policy: A policy can be defined as a way how an agent behaves at a given time. It maps the perceived states of the environment to the actions taken on those states. A **policy is the core element of the RL as it alone can define the behavior of the agent**. In some cases, **it may be a simple function or a lookup table, whereas, for other cases, it may involve general computation as a search process**. It could be deterministic or a stochastic policy:

For deterministic policy: $a = \pi(s)$

For stochastic policy: $\pi(a | s) = P[A_t = a | S_t = s]$

2) Reward Signal: The goal of reinforcement learning is defined by the reward signal. At each state, the environment sends an immediate signal to the learning agent, and this signal is known as a reward signal. These rewards are given according to the good and bad actions taken by the agent. **The agent's main objective is to maximize the total number of rewards for good actions**. The reward signal can change the policy, such as if an action selected by the agent leads to low reward, then the policy may change to select other actions in the future.

3) Value Function: The value function **gives information about how good the situation and action are and how much reward an agent can expect**. A reward indicates the immediate signal for each good and bad action, whereas a value function specifies the good state and action for the future. The value function depends on the reward as, without reward, there could be no value. The goal of estimating values is to achieve more rewards.

4) Model: The **last element of reinforcement learning is the model**, which **mimics the behavior of the environment**. With the help of the model, one can make inferences about how the environment will behave. Such as, if a state and an action are given, then a model can predict the next state and reward.

The model is used for planning, which means it provides a way to take a course of action by considering all future situations before actually experiencing those situations. The approaches for solving the RL problems with the help of the model are termed as the model-based approach. Comparatively, an approach without using a model is called a model-free approach.

How to formulate a basic Reinforcement Learning problem?

Some key terms that describe the basic elements of an RL problem are:

- **Agent():** An entity that can perceive/explore the environment and act upon it.
- **Environment():** A situation in which an agent is present or surrounded by. In RL, we assume the stochastic environment, which means it is random in nature.
- **Action():** Actions are the moves taken by an agent within the environment.
- **State():** State is a situation returned by the environment after each action taken by the agent.
- **Reward():** A feedback returned to the agent from the environment to evaluate the action of the agent.
- **Policy():** Policy is a strategy applied by the agent for the next action based on the current state.
- **Value():** Future reward that an agent would receive by taking an action in a particular state. It is expected long-term return with the discount factor and opposite to the short-term reward.
- **Q-value():** It is mostly similar to the value, but it takes one additional parameter as a current action.

An RL problem can be best explained through games. Let's take the game of PacMan where the goal of the agent (PacMan) is to eat the food in the grid while avoiding the ghosts on its way. In this case, the grid world is the interactive environment for the agent where it acts. Agent receives a reward for eating food and punishment if it gets killed by the ghost (loses the game). The states are the location of the agent in the grid world and the total cumulative reward is the agent winning the game.

In order to build an optimal policy, the agent faces the dilemma of exploring new states while maximizing its overall reward at the same time. This is called Exploration vs Exploitation trade-off. To balance both, the best overall strategy may involve short term sacrifices. Therefore, the agent should collect enough information to make the best overall decision in the future.

Markov Decision Processes (MDPs) Markov Decision Process (MDP's) – are mathematical frameworks to describe an environment in RL and almost all RL problems can be formulated using MDPs. The set of parameters that include Set of finite states – S , Set of possible Actions in each state – A , Reward – R , Model – T , Policy – π . The outcome of deploying an action to a state doesn't depend on previous actions or states but on current action and state. However, real world environments are more likely to lack any prior knowledge of environment dynamics. Model-free RL methods come handy in such cases.

States:	S
Model:	$T(S, a, S') \sim P(S' S, a)$
Actions:	$A(S), A$
Reward:	$R(S), R(S, a), R(S, a, S')$
Policy:	$\pi(S) \rightarrow a$ π^*

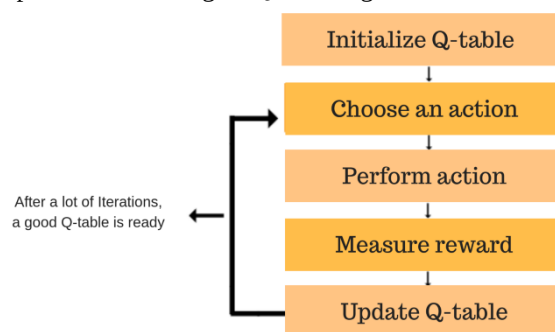
Markov Decision Process

Some of the most used Reinforcement Learning algorithms

Reinforcement learning algorithms are mainly used in AI applications and gaming applications. The main used algorithms are:

Q-Learning: it's a value-based model free approach for supplying information to intimate which action an agent should perform. It revolves around the notion of updating Q values which shows the value of doing action A in state S. Value update rule is the main aspect of the Q-learning algorithm.

- Q-learning is an off policy RL algorithm, which is used for the temporal difference Learning. The temporal difference learning methods are the way of comparing temporally successive predictions.
- It learns the value function $Q(S, a)$, which means how good to take action "a" at a particular state "s".
- The below flowchart explains the working of Q-learning:



Q-learning can be used for building a self-playing PacMan agent. It revolves around the notion of updating Q values which denotes value of performing action a in state s . The following value update rule is the core of the Q-learning algorithm.

$$Q(s_t, a_t) \leftarrow (1 - \alpha) \cdot \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)$$

learned value

State Action Reward State action (SARSA):

- SARSA stands for **State Action Reward State action**, which is an on-policy temporal difference learning method. The on-policy control method selects the action for each state while learning using a specific policy.
- The goal of SARSA is to calculate the $Q, \pi(s, a)$ for the selected current policy π and all pairs of (s, a) .
- The main difference between Q-learning and SARSA algorithms is that unlike Q-learning, the maximum reward for the next state is not required for updating the Q-value in the table.

- In SARSA, new action and reward are selected using the same policy, which has determined the original action.
- The SARSA is named because it uses the quintuple $Q(s, a, r, s', a')$. Where,
 s : original state
 a : Original action
 r : reward observed while following the states
 s' and a' : New state, action pair.

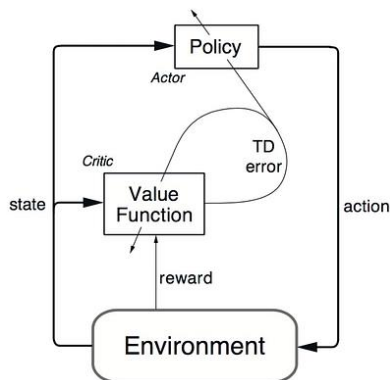
Deep Q Neural Network (DQN):

- As the name suggests, DQN is a Q-learning using Neural networks.
- For a big state space environment, it will be a challenging and complex task to define and update a Q-table.
- To solve such an issue, we can use a DQN algorithm. Where, instead of defining a Q-table, neural network approximates the Q-values for each action and state.

Q-learning and SARSA (State-Action-Reward-State-Action) are two commonly used RL algorithms. They differ in terms of their exploration strategies while their exploitation strategies are similar. While *Q-learning is an off-policy method* in which the agent learns the value based on action a^* derived from another policy, *SARSA is an on-policy method* where it learns the value based on its current action a derived from its current policy. *These two methods are simple to implement but lack generality as they do not have the ability to estimate values for unseen states.*

This can be overcome by **more advanced algorithms such as Deep Q-Networks(DQNs) which use Neural Networks to estimate Q-values. But DQNs can only handle discrete, low-dimensional action spaces.**

Deep Deterministic Policy Gradient (DDPG) is a model-free, off-policy, actor-critic algorithm that tackles this problem by learning policies in high dimensional, continuous action spaces. The figure below is a representation of actor-critic architecture.



Practical applications of Reinforcement Learning

Since, RL requires a lot of data, therefore it is most applicable in domains where simulated data is readily available like gameplay, robotics.

- Autonomous Self Driving Cars
- Machine Learning and Data Processing
- Training system which would issue custom instructions and materials with respect to the requirements of students
- AI Toolkits, Manufacturing, Automotive, Healthcare, and Bots
- Aircraft Control and Robot Motion Control
- Building artificial intelligence for computer games
- RL is quite widely used in building AI for playing computer games. AlphaGo Zero is the first computer program to defeat a world champion in the ancient Chinese game of Go. Others include ATARI games, Backgammon ,etc
- In robotics and industrial automation, RL is used to enable the robot to create an efficient adaptive control system for itself which learns from its own experience and behavior. DeepMind's work on Deep Reinforcement Learning for Robotic Manipulation with Asynchronous Policy updates is a good example of the same. Watch this interesting demonstration video.

- Other applications of RL include abstractive text summarization engines, dialog agents (text, speech), gameplays which can learn from user interactions and improve with time, learning optimal treatment policies in healthcare and RL based agents for online stock trading.

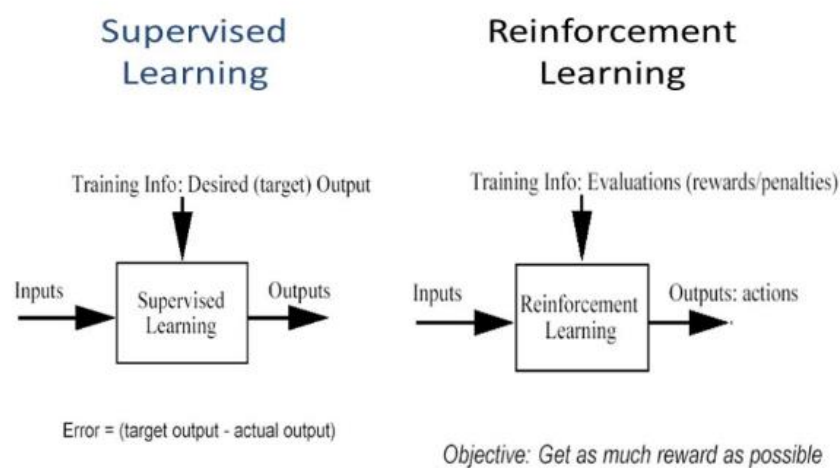
Purpose oriented applications in specific field as:

- Robotics:** RL is used in Robot navigation, Robo-soccer, walking, juggling, etc.
- Control:** RL can be used for adaptive control such as Factory processes, admission control in telecommunication, and Helicopter pilot is an example of reinforcement learning.
- Game Playing:** RL can be used in Game playing such as tic-tac-toe, chess, etc.
- Chemistry:** RL can be used for optimizing the chemical reactions.
- Business:** RL is now used for business strategy planning.
- Manufacturing:** In various automobile manufacturing companies, the robots use deep reinforcement learning to pick goods and put them in some containers.
- Finance Sector:** The RL is currently used in the finance sector for evaluating trading strategies.

Difference between Reinforcement Learning and Supervised Learning

The Reinforcement Learning and Supervised Learning both are the part of machine learning, but both types of learnings are far opposite to each other. The RL agents interact with the environment, explore it, take action, and get rewarded. Whereas supervised learning algorithms learn from the labeled dataset and, on the basis of the training, predict the output.

In supervised learning, the model is trained with a training dataset that has a correct answer key. The decision is done on the initial input given as it has all the data that's required to train the machine. The decisions are independent of each other so each decision is represented through a label. Example: Object Recognition



Difference between Supervised and Reinforcement Learning

In reinforcement learning, there isn't any answer and the reinforcement agent decides what to be done to perform the required task. As the training dataset isn't available, the agent had to learn from its experience. It's all about compiling the decisions in a sequential manner. To be said in simpler words, the output relies on the current input state and the next input relies on the output of the previous input. We give labels to the sequence of dependent decisions. Decisions are dependent. Example: Chess Game

The difference table between RL and Supervised learning is given below:

Reinforcement Learning	Supervised Learning
RL works by interacting with the environment.	Supervised learning works on the existing dataset.
The RL algorithm works like the human brain works when making some decisions.	Supervised Learning works as when a human learns things in the supervision of a guide.
There is no labeled dataset is present	The labeled dataset is present.

No previous training is provided to the learning agent.	Training is provided to the algorithm so that it can predict the output.
RL helps to take decisions sequentially.	In Supervised learning, decisions are made when input is given.

Advantages of Reinforcement learning

1. Reinforcement learning can be used to solve very complex problems that cannot be solved by conventional techniques.
2. The model can correct the errors that occurred during the training process.
3. In RL, training data is obtained via the direct interaction of the agent with the environment
4. Reinforcement learning can handle environments that are non-deterministic, meaning that the outcomes of actions are not always predictable. This is useful in real-world applications where the environment may change over time or is uncertain.
5. Reinforcement learning can be used to solve a wide range of problems, including those that involve decision making, control, and optimization.
6. Reinforcement learning is a flexible approach that can be combined with other machine learning techniques, such as deep learning, to improve performance.

Disadvantages of Reinforcement learning

1. Reinforcement learning is not preferable to use for solving simple problems.
2. Reinforcement learning needs a lot of data and a lot of computation
3. Reinforcement learning is highly dependent on the quality of the reward function. If the reward function is poorly designed, the agent may not learn the desired behavior.
4. Reinforcement learning can be difficult to debug and interpret. It is not always clear why the agent is behaving in a certain way, which can make it difficult to diagnose and fix problems.